

LECTURE 1: INTRODUCTION TO AGENTS AND MAS

Introduction to Multi-Agent Systems (MESIIA, MIA)

Department of Computer Science and Mathematics

Rovira i Virgili University

Fall 2019

Outline of the lecture

- Main trends in Computer Science
 - Agents and multi-agent systems
 - Objections to MAS
 - Agents vs. related technologies
 - Viewpoints on agent technology
 - Pitfalls of agent development
 - Challenges on agent technology
-

Important aspects in Computer Science

- Five ongoing trends are marking the history of computing:
 - *ubiquity*
 - *interconnection*
 - *intelligence*
 - *delegation*
 - *human-orientation in programming methodologies*

Ubiquity



- The continuous reduction in size and cost of computing capability has made it possible to introduce processing power into places and devices that would have once been uneconomic / unimaginable [e.g. Nike+]
 - As processing capability spreads, sophistication and intelligence become ubiquitous
-

Ambient Intelligence

- Aimed at seamless delivery of services and applications.
 - Relies on ubiquitous computing, ubiquitous communication and intelligent user interfaces.
 - The vision
 - An environment of potentially **thousands of embedded and mobile devices** (or software components) interacting to support user-centered goals and activity.
 - **Autonomy, distribution, adaptation** and **responsiveness** are key characteristics of these independent and distributed components, and in this sense they share the same characteristics as agents.
 - Requires agents to be able to interact with numerous other agents in the environment around them in order to achieve their goals.
-

Transportation
Intelligent Station



Commerce
Intelligent Exhibition



Leisure
Intelligent Playground



Education
Intelligent Classroom



Home
Intelligent Living-room



Work
Intelligent Office

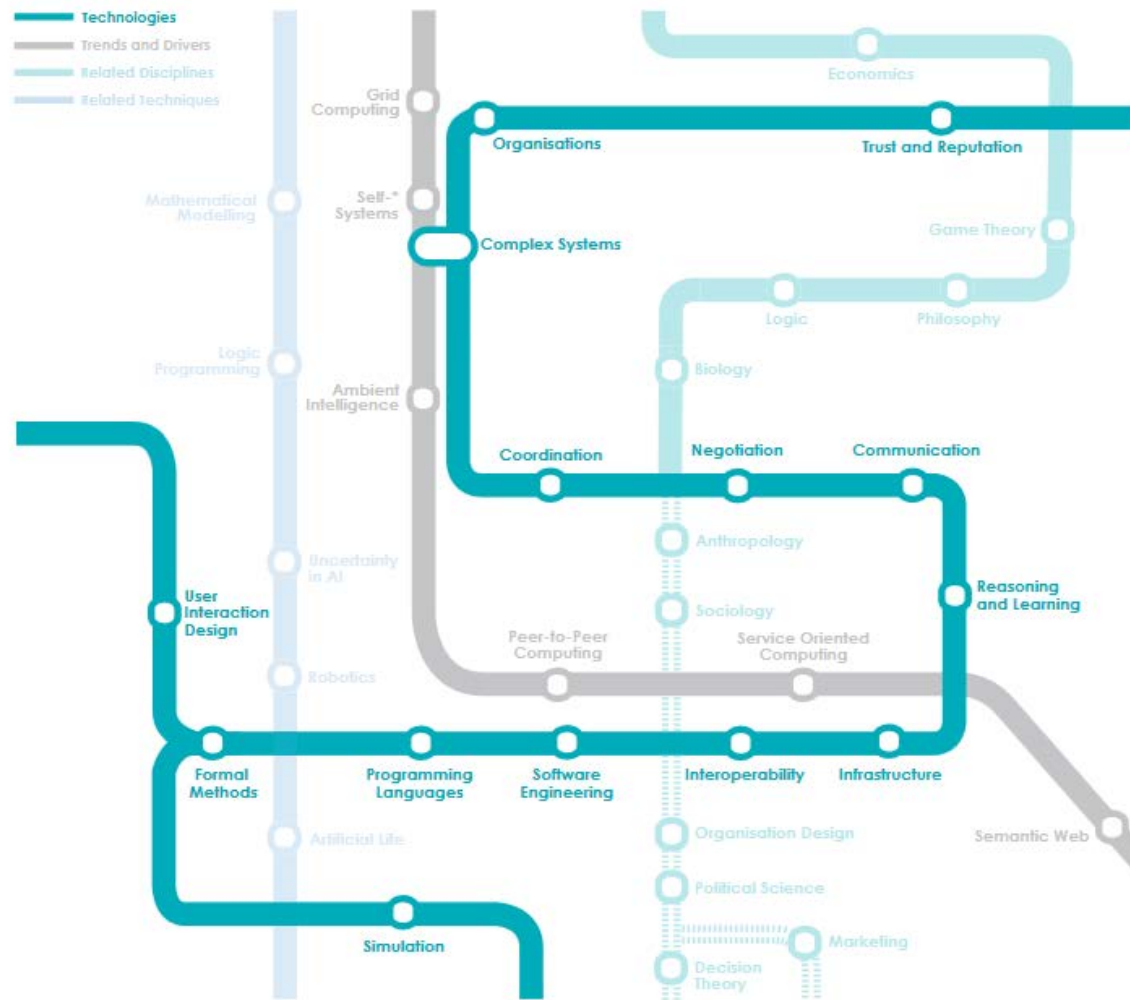


Interconnection

- Computer systems today no longer stand alone, but are networked into large distributed systems
 - Obvious example: Internet
 - Since distributed and concurrent systems have become the norm, some researchers are putting forward theoretical models that portray *computing as primarily a process of interaction*
-

Agent Technology: Computing as Interaction

A Roadmap for Agent Based Computing



Compiled, written and edited by

Michael Luck, Peter McBurney, Onn Shehory, Steve Willmott and the AgentLink Community

Connectivity -Technological context

- Semantic Web

- The Semantic Web is based on the idea that the data on the Web can be defined and linked in such a way that it can be used by machines for the automatic processing and integration of data across different applications (Berners-Lee et al., 2001).

- Peer-to-Peer computing

- Grid computing

Grid Computing

What is all about Grid ?

- 1) Sharing, Selecting, aggregation of resources with a policy universally agreed.
- 2) Distributive supercomputing for a better future



Copyright: adasik@oic

Intelligence

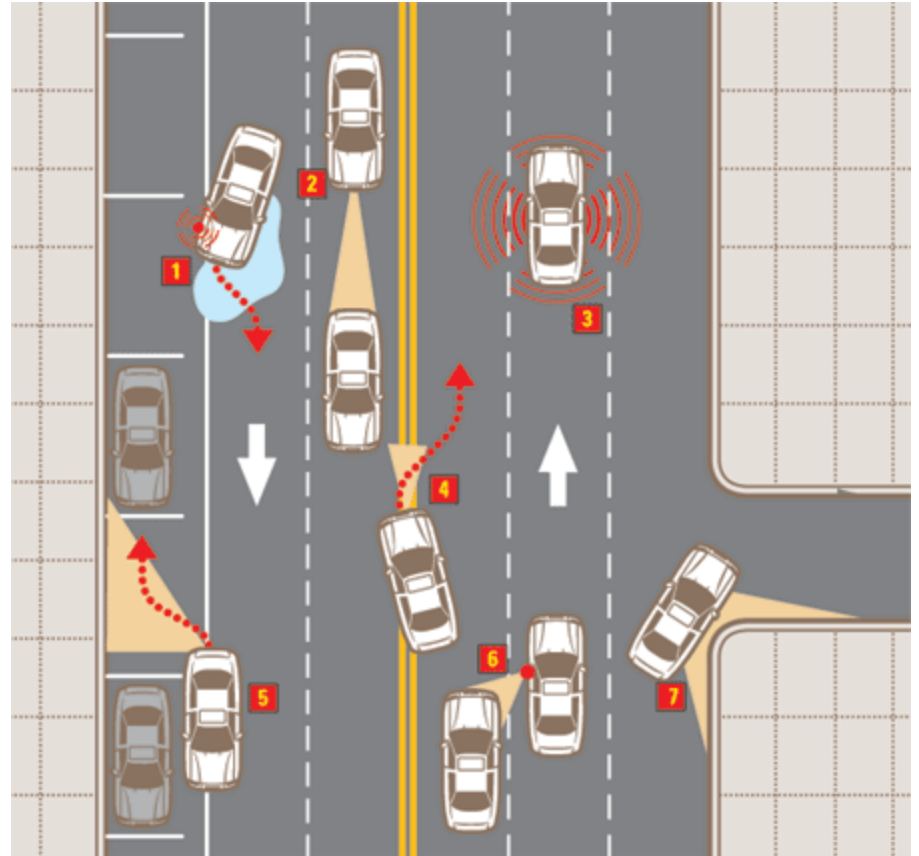
- The **complexity** of tasks that we are capable of automating and delegating to computers has grown steadily
 - That requires an increasing amount of domain knowledge and inference capabilities, in short, more **intelligent** problem solving techniques
-

Delegation

- Computers are doing more for us – without our intervention
 - We are *giving control* to computers, even in safety critical tasks
 - One example: fly-by-wire aircraft, where the machine's judgment may be trusted more than an experienced pilot
 - Next on the agenda: self-driving cars, intelligent braking systems, cruise control that maintains distance from car in front...
-

Driving automation

1. Road Condition Reporting
2. Adaptive Cruise Control
3. Omnidirectional Collision System
4. Lane-Departure Prevention
5. Auto Parallel Park
6. Blind-Spot Sensors
7. Corner Speed



Human-Oriented Programming

- The movement away from machine-oriented views of programming toward concepts and metaphors that more closely reflect the way we ourselves understand the world
 - Programmers (and users!) relate to the machine differently
 - Programmers conceptualize and implement software in terms of higher-level – more human-oriented – abstractions
-

Agent-Oriented Software Engineering

- Programming has progressed through
 - machine code
 - assembly language
 - machine-independent programming languages
 - sub-routines
 - procedures & functions
 - abstract data types
 - objectsto *agents*

Where does it bring us?

- *Delegation* and *Intelligence* imply the need to build computer systems that can act effectively on our behalf
- This implies:
 - The ability of computer systems to act *independently*
 - The ability of computer systems to act in a way that *represents our best interests* while interacting with other humans or systems

Interconnection and Distribution

- *Interconnection* and *Distribution* have become core motifs in Computer Science
 - But Interconnection and Distribution, coupled with the need for systems to represent our best interests, implies systems that can *cooperate* and *reach agreements* (or even *compete*) with other systems that have different interests (much as we do with other people)
-

So Computer Science expands...

- These issues were not studied in Computer Science until recently
- All of these trends have led to the emergence of a new field in Computer Science:
multiagent systems

Agents, a Definition

- An agent is a computer system that is located in a dynamic environment and is capable of *independent* action on behalf of its user or owner (figuring out what needs to be done to satisfy design objectives, rather than constantly being told)
- *Intelligent* action, probably using Artificial Intelligence tools and techniques
- An agent should satisfy a certain number of properties [lecture 3]

Multiagent Systems, a Definition

- A multiagent system is one that consists of a number of agents, which *interact* with one-another
- In the most simple case, all agents are programmed by the same team and they collaborate to complete a task
- In the most general case, agents will be acting on behalf of users with different goals and motivations
- To successfully interact, they will require the ability to *cooperate*, *coordinate*, and *negotiate* with each other, much as people do



Related areas

- ❑ Several sub-disciplines of information technology are related to software agent technology:
 - computer networks,
 - software engineering,
 - Artificial Intelligence,
 - human-computer interaction,
 - distributed and concurrent systems,
 - ...
-

Micro and macro aspects

- Agent technologies can be grouped into three categories
 - Agent-level [micro level]
 - technologies and techniques concerned only with individual agents — for example, procedures for agent reasoning and learning [Agent architectures – lecture 2]
 - Interaction-level
 - technologies and techniques that concern the communication between agents
 - communication languages, interaction protocols and resource allocation mechanisms [Agent communication-lecture 4. Coordination and cooperation mechanisms-lectures 5 to 8]
 - Organization-level [macro level]
 - technologies and techniques related to agent societies as a whole
 - structure, trust, norms, obligations, etc. [Agent societies-lecture 9]

Multiagent Systems

- In Multiagent Systems, the main questions we address are:
 - What kinds of languages can agents use to **communicate**?
 - How can **cooperation** emerge in societies of self-interested agents?
 - How can self-interested agents recognize conflict, and how can they (nevertheless) reach **agreement**?
 - How can autonomous agents **coordinate** their activities so as to cooperatively achieve goals?

Multiagent Systems

- While these questions are all addressed in part by other disciplines (notably Economics and Social Sciences), what makes the multiagent systems field unique is that it emphasizes that the agents in question are *computational, information processing* entities.
-

Example 1 - Spacecraft Control

- When a space probe makes a flight to the outer planets, a ground crew is usually required to continually track its progress, and decide how to deal with unexpected eventualities. This is costly and, if decisions are required *quickly*, it is simply not practicable. NASA is investigating the possibility of making probes more *autonomous* — giving them richer decision making capabilities and responsibilities.
 - *This is not fiction: NASA's DS1 has done it!*
-



- + NASA Home
- + Ames Home
- + Intelligent Systems
Division

Remote Agent

+ HOME

+ FAQ

+ BENEFITS

+ STATUS

+ HISTORY

+ HOW IT WORKS

+ MISSIONS

+ EXPERIMENT

+ IN THE MEDIA

+ TEAM

+ PUBLICATIONS

Search

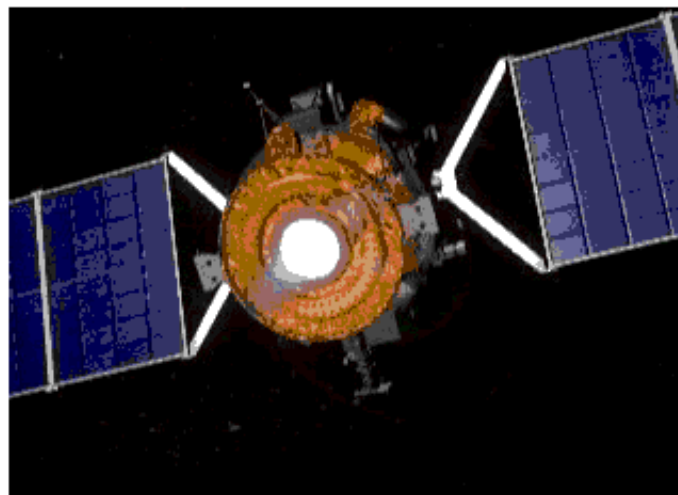
Intelligent Systems Division

GO

OVERVIEW

1999 Co-Winner of NASA's Software of the Year Award

It's one small step in the history of space flight. But it was one giant leap for computer-kind, with a state of the art artificial intelligence system being given primary command of a spacecraft. Known as Remote Agent, the software operated NASA's Deep Space 1 spacecraft and its futuristic ion engine during two experiments that started on Monday, May 17, 1999. For two days Remote Agent ran on the on-board computer of Deep Space 1, more than 60,000,000 miles (96,500,000 kilometers) from Earth. The tests were a step toward robotic explorers of the 21st century that are [less costly](#), [more capable](#) and [more independent](#) from ground control.



A second remote agent experiment was conducted on Friday, May 21, starting at 7:15 a.m. Pacific Time, accomplishing all of the remaining test objectives of the remote agent team. For more information, see Friday afternoon's [status](#) message and transcript of the Remote Agent

FEATURES

Current Status

Notes on the progress of the experiment.

DS1 Mission

More information on DS1 can be found at their [website](#).

Progress Display

A java applet which shows the remote agent's activities onboard the spacecraft.

DS1 Animation

A short computer graphics animation created for the news media.

Activities

Several activities related to the experiment

3D Spacecraft Model

VRML plug in required.

Autonomous Agents for specialized tasks

- The DS1 example is one of a generic class
 - Agents (and their physical instantiation in robots) have a role to play in high-risk situations, unsuitable or impossible for humans
 - The degree of autonomy will differ depending on the situation (remote human control may be an alternative, but not always)
-

Example 2 - Air Traffic Control

“A key air-traffic control system suddenly fails, leaving flights in the vicinity of the airport with no air-traffic control support. Fortunately, autonomous air-traffic control systems in nearby airports recognize the failure of their peer, and cooperate to track and deal with all affected flights.”

- Systems **taking the initiative** when necessary
 - Agents **cooperating** to solve problems beyond the capabilities of any individual agent
-

Example 3 - Internet Agents

- Searching the Internet for the answer to a specific query can be a long and tedious process. So, why not allow a computer program — an agent — do searches for us? The agent would typically be given a query that would require synthesizing pieces of information from various different Internet information sources.

[Lecture 3 on Agent Types, including Information Agents]

Example 4 – e-personal assistants

- Internet agents need not simply search
 - They can plan, arrange, buy, negotiate – carry out arrangements of all sorts that would normally be done by their human user
 - As more can be done electronically, software agents theoretically have more access to systems that affect the real-world
 - For example, preparing a holiday trip
-

Research Issues

- How do you state your **preferences** to your agent?
 - How can your agent compare different deals from different vendors? What if there are many different parameters?
 - What algorithms can your agent use to **negotiate** with other agents?
 - These issues aren't frivolous – automated procurement could be used massively by (for example) government agencies
-

Multiagent Systems is Interdisciplinary

- The field of Multiagent Systems is influenced and inspired by many other fields:
 - Economics
 - Philosophy
 - Game Theory
 - Logic
 - Ecology
 - Social Sciences
- This can be both a strength (infusing well-founded methodologies into the field) and a weakness (there are many different views as to what the field is about)
- This has analogies with Artificial Intelligence itself

Objections to MAS (I)

- Isn't it all just Distributed/Concurrent Systems?
There is much to learn from this community, but:
 - Agents are assumed to be autonomous, capable of making independent decision – so they need mechanisms to synchronize and coordinate their activities **at run time**
 - Agents are (can be) self-interested, so their interactions are “economic” encounters (negotiation)
-

Objections to MAS (II)

■ Isn't it all just AI?

- We don't need to solve all the problems of Artificial Intelligence (i.e., all the components of intelligence) in order to build really useful agents
- Classical AI ignored *social* aspects of agency. These are important parts of intelligent activity in real-world settings

Objections to MAS (III)

■ Isn't it all just Economics/Game Theory?

These fields also have a lot to teach us in multiagent systems, but:

- Insofar as game theory provides *descriptive* concepts, it doesn't always tell us *how* to compute solutions; we're concerned with computational, resource-bounded agents
- Some assumptions in economics/game theory (such as a *rational agent*) may not be valid or useful in building artificial agents

Objections to MAS (IV)

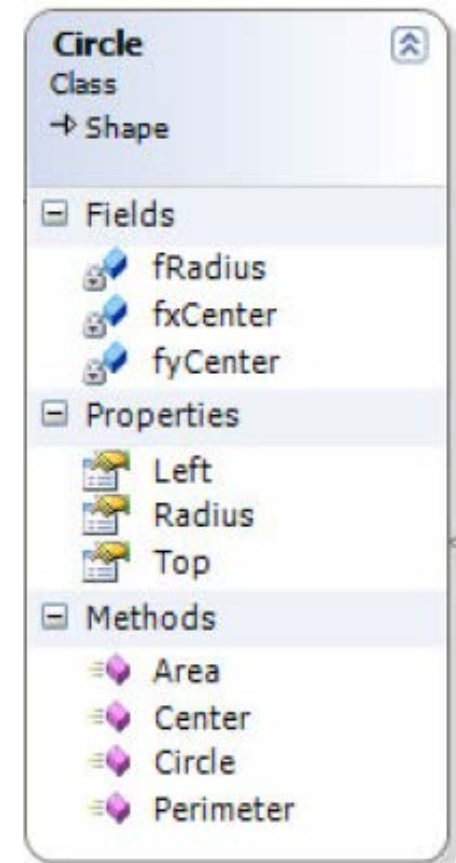
- Isn't it all just Social Science?
 - We can draw insights from the study of human societies, but there is no particular reason to believe that artificial societies will be constructed in the same way
 - Again, we have inspiration and cross-fertilization, but hardly subsumption
-

Agents versus related technologies

- If agents are autonomous entities that display an intelligent behaviour, what makes them so different from other well known techniques, like object-oriented programming or intelligent (knowledge-based) systems?

Agents and Objects

- Are agents just **objects** by another name?
- Object:
 - encapsulates some state
 - communicates via message passing
 - has methods, corresponding to operations that may be performed on the state



Agents vs Objects (I)

- ❑ *Agents are **autonomous***
 - Agents embody a stronger notion of autonomy than objects
 - They decide for themselves whether or not to perform an action on request from another agent
 - When a method is invoked on an object, it is always executed

Agents vs Objects (II)

- *Agents are **smart, intelligent***

- Capable of flexible (reactive, proactive, social) behaviour
- The standard object model has nothing to say about such types of behaviour

- *Agents are **active***

- A multi-agent system is inherently multi-threaded, in that each agent is assumed to have at least one thread of active control

Agents vs Objects (III)

Objects

- Encapsulate state and control over it via **methods**
- **Passive** – have no control over a method execution
- **Non autonomous**
- **Reactive to events**

Agents

- Encapsulate state, control over it via ***actions and goals***
- ***Active*** – can decide when to act and how
- ***Autonomous***
- ***Proactive***

In summary ...

- *Objects do it for free*

An object cannot refuse a method invocation

- *Agents do it because they want to*

The service requester is authorised, the agent has enough resources available, the action is convenient for the agent, ...

- *Agents do it for money*

The agent can get an economic profit

Agents and Expert Systems

- Aren't agents just **expert systems** by another name?
- Expert systems contain typically disembodied 'expertise' about some (abstract) domain of discourse (e.g. blood diseases)
- Example: MYCIN knows about blood diseases in humans
 - It has a wealth of knowledge about blood diseases, in the form of rules
 - A doctor can obtain expert advice about blood diseases by giving MYCIN facts, answering questions, and posing queries

Agents and Expert Systems

- Main differences:
 - agents *situated in an environment*:
MYCIN is not aware of the world — the only information that it obtains is by asking questions to the user
 - agents *act*:
MYCIN does not operate on patients
 - Expert systems typically work alone, whereas agents *cooperate* to achieve a common goal.
- Sometimes an expert system is *agentified* and included in a MAS

Some Views of the Field

- *Agents as a paradigm for software engineering:*

Software engineers have derived a progressively better understanding of the characteristics of complexity in software. It is now widely recognized that *interaction* is probably the most important single characteristic of complex software

Agents as design metaphor

- **Agent-oriented software engineering**
 - Agents allow software designers and developers to structure an application using autonomous, communicative components.
 - In this sense, software agents offer a new and often more appropriate route to the development of complex computational systems, especially in open and dynamic environments.
 - Modularity, reusability, collaborative behaviour



Agents as source of technologies

- Software agent technologies span a range of specific **techniques and algorithms**
 - balancing **reaction** and **deliberation** in individual agent architectures **[lecture 2]**
 - **learning** from and about other agents in the environment
 - eliciting and acting upon **user preferences**
 - finding ways to **negotiate** and **cooperate** with other agents
 - developing appropriate means of forming and managing **coalitions** (and other organizations)

Social simulation

- *Agents as a tool for understanding human societies:*

Multiagent systems provide a new tool for **simulating societies**, which may help shed some light on various kinds of social processes.

Agent-based simulation

- Multi-agent systems provide strong models for representing complex and dynamic real-world environments.
 - modelling of the impact of climate change on biological populations
 - modelling the impact of public policy options on social or economic behaviour
 - modelling intelligent buildings
 - modelling traffic systems
 - modelling biological populations
-

Multi-agent Modeling

DEMONSTRATIONS

Demonstrations 1 - 18 of 18

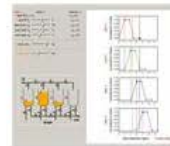
 [Subscribe to RSS feed](#)



Exchange Networks



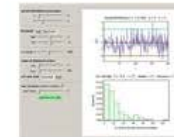
Fishing with Long Line or Gill Net?



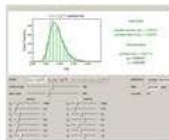
Estimating Processing Times in a Row of Storage Tanks



Ant Colony Optimization (ACO)



Estimating the Time between Mishaps from Quality Control Data



Assessing Total Risk from Interacting Factors



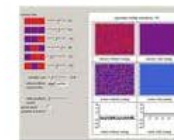
Boids: Simulated Flocking Behavior



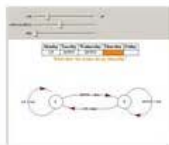
Mean Distance of Agents in Diffusion-Limited Aggregation



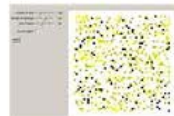
Sugarscape: Agent-Based Modeling



Emulating Land Use Evolution with a Cellular Automaton



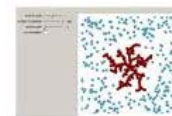
Trader Dynamics in Minimal Models of Financial Complexity



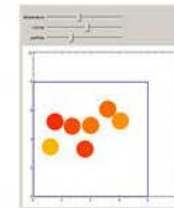
Garbage Collection by Ants



Predator-Prey Ecosystem: A Real-Time Agent-Based Simulation



Diffusion-Limited Aggregation: A Real-Time Agent-Based Simulation



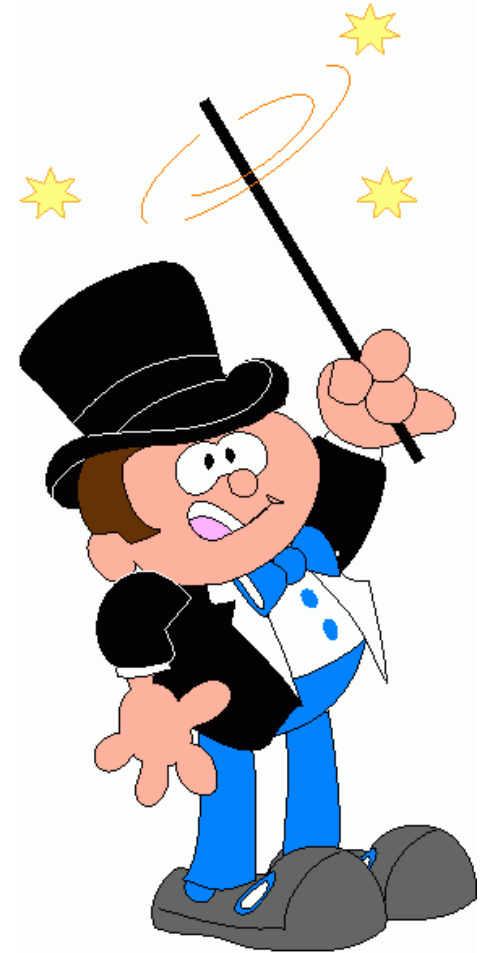
Molecular Dynamics of Lennard-Jones Particles Using the Velocity Verlet Algorithm

Pitfalls of Agent Development

- Agents have many interesting and positive properties, but ...
 - It is **difficult** to design, implement, deploy and maintain a MAS
 - There aren't any firmly established **agent-oriented software engineering methodologies**
 - There are several potential **problems** that should be carefully considered when starting an agent-based approach
-

Overselling Agents

- Agents are *not* equal to AI!
- Agents *may* make it easier to solve certain classes of problems...but they do not make the impossible possible.
- No evidence that any system developed using agent technology could not have been built just as easily using non-agent techniques

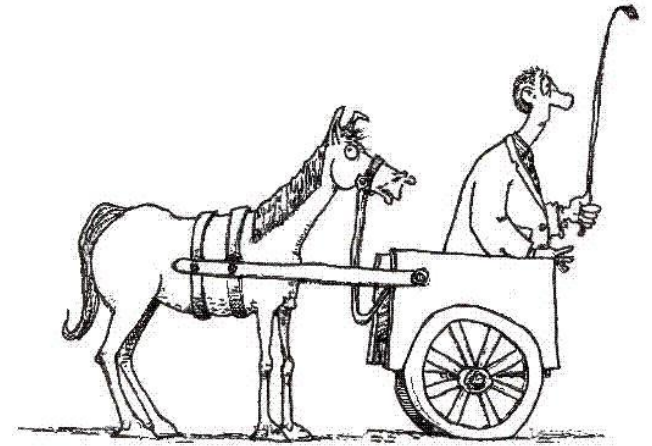


Universal Solution?

- Agents have been used in a wide range of applications, but they are *not* a universal solution
 - For many applications, conventional software paradigms (e.g., OO) can be more appropriate
 - Given a problem for which an agent and a non-agent approach appear equally good, prefer the non-agent solution!
 - In summary: danger of believing that agents are the right solution to *every* problem
-

Don't Know What Agents Are Good For

- Having developed some agent technology, you **search for an application** to use them
- Putting the cart before the horse!
- The lesson: be sure you understand **how** and **where** your new technology may be most usefully applied
- Do not attempt to apply it to arbitrary problems and resist temptation to apply it to every problem



Confuse Prototypes with Systems

- Prototypes are easy (particularly with nice GUI builders!)
 - Field-tested production systems are hard
 - For instance, how will the agent-based software be maintained? (e.g. in a hospital)
 - The process of scaling up from a single-machine multi-threaded Java application to a multi-user distributed system is much harder than it appears
-

A Silver Bullet for Soft. Eng. (I)

- The holy grail of software engineering is a “*silver bullet*”: an order of magnitude improvement in software development
- Technologies promoted as the silver bullet:
 - ❑ COBOL
 - ❑ Structured programming
 - ❑ Graphical/visual programming
 - ❑ Object Oriented Programming
 - ❑ Java



A Silver Bullet for Soft. Eng. (II)

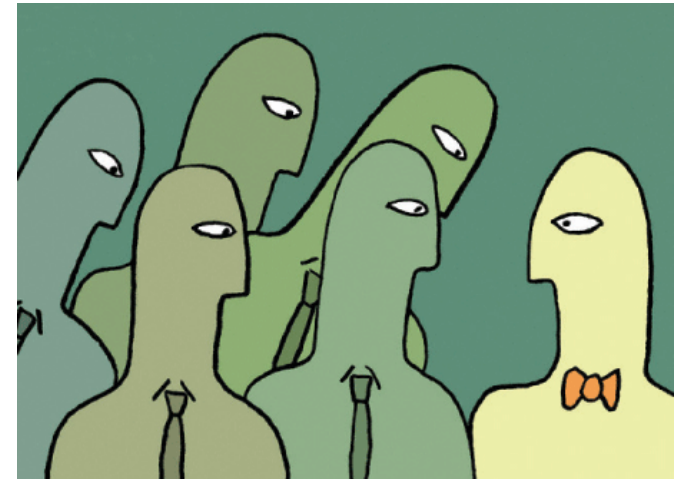
- Agent technology is *not* a silver bullet
 - It is true that there are good reasons to believe that agents are a *useful* way of tackling some problems ...
 - ... but these arguments remain largely *untested in practice*
-

Don't Forget - It's Distributed (I)

- **Distributed systems** = one of the most complex classes of computer systems to design and implement
 - Not only different **modules**, but the **interconnection** between them
 - Multi-agent systems tend to be distributed!
 - Problems of distribution do not go away, just because a system is agent-based
-

Don't Forget - It's Distributed (II)

- A typical multi-agent system will be *more complex* than a typical distributed system
 - Autonomous entities
 - Conflicts between entities
 - Dynamic forms of cooperation
 - Emergent behaviour
- Recognize distributed systems problems
- Make use of DS-DAI expertise



Don't exploit concurrency

- In a poor MAS design the amount of **concurrent problem solving** is small or inexistent.
- **Serial processing** in distributed system!
 - ❑ Agent A makes a task, sends results to B
 - ❑ Agent B makes another task, sends results to C ...
- If you don't exploit concurrency, why have an agent solution?



Use Too Much AI

- Temptation to focus on the agent-specific aspects of the application
 - Result: an agent framework too overburdened with **experimental AI techniques** to be usable
 - Resist the temptation to believe that all AI features are essential in your agent system
 - The lesson: build agents with a minimum of AI; as success is obtained with such systems, progressively evolve them into richer systems
-

Not Enough AI

- Don't call your on-off switch an agent!
- Be realistic: it is becoming common to find everyday distributed systems referred to as MAS
- Problems:
 - leads to the term “agent” *losing any meaning*
 - raises expectations of software recipients
 - leads to cynicism on the part of software developers

Too Many Agents

- Individual agents don't have to be very complex to generate complex behaviour at the system level
- A large number of agents:
 - *Can lead to interesting and unexpected emergent functionality, but ...*
 - *... also to **chaotic behaviour***
- Lessons
 - Keep interactions to a minimum
 - Keep protocols simple



Too few agents

- Some designers imagine a separate agent for every possible task
 - Others don't recognize value of a multi-agent approach at all
 - One “all powerful” centralised planning and controller agent
 - Result is like OO program with 1 class
 - Lesson: choose agents of the right size
-

System is anarchic

- Cannot simply bundle a group of agents together (except possibly in simulations)
 - Most agent systems require **system-level engineering**
 - For large systems, or for systems in which the society is supposed to act with some commonality of purpose, this is particularly important
 - **Organization structure** (even in the form of formal communication channels) is essential
-

Ignore Available Standards

- There are no established agent standards
- Developers often believe they have no choice but to design and build all agent-specific components from scratch
- But there are some **de facto** standards
- Examples:
 - ❑ CORBA – communication middleware
 - ❑ OWL – ontology language
 - ❑ FIPA agent architecture
 - ❑ FIPA-ACL – agent communication language

Challenges in agent technology

- Implementing multi-agent systems is still a complex task.
 - ❑ Lack of maturity in both methodologies and programming tools.
 - ❑ Lack of specialized debugging tools.
 - ❑ Lack of skills needed to move from analysis and design to code.
 - ❑ Problems associated with awareness of the specifics of different agent platforms.
-

Achieve industrial quality (I)

- Agent oriented design methodologies, tools and development environments.
 - Analysis point of view - dedicated concepts and languages:
 - Dynamic aspects, such as time and action.
 - Locality aspects, such as position in a space.
 - Design point of view - reusability, libraries of
 - Generic organisation models (hierarchical, flat, etc.).
 - Generic agent models (reactive, deliberative, etc.)
 - Generic task models (diagnostic, information filtering, etc.).
 - Communication languages and patterns for agent societies.
 - Interaction protocol patterns between agents with special roles.

Achieve industrial quality (II)

- Seamless integration with existing technologies.
 - Web, information systems
 - Non-functional properties: scalability, performance, reliability, and robustness.
-

Reasoning capabilities

- Distributed planning techniques
 - Negotiation and argumentation strategies
 - Coalition formation
 - Dynamic coalitions of small groups of agents.
 - Virtual organisations, e-institutions
 - Heterogeneity of agents, trust, and failure handling and recovery.
 - Norms, legislation, authorities, and enforcement.
-

Adapt to user requirements

- User profiling
 - Personalisation
 - Adapt to changes in environment
 - Learning
 - Evolutionary programming techniques
-

User confidence and trust (I)

- Security technologies.
 - Security in open agent systems.
 - Agents must be secure and must not reveal information inappropriately.
 - Confidence that agents representing other parties will behave within certain constraints.
-

User confidence and trust (II)

- Models and infrastructure for trust and reputation.
 - **Reputation** mechanisms to assess the past behaviour of particular agents or users, to allow avoidance of untrustworthy agents.
 - The adoption of **norms** (social rules) by all members of an open agent system.
 - Enforcement of **sanctions** against agents not following rules (or self-enforcing protocols).
 - The use of **electronic contracts** to represent and enforce agreements between several parties.
-

Exiting out of labs

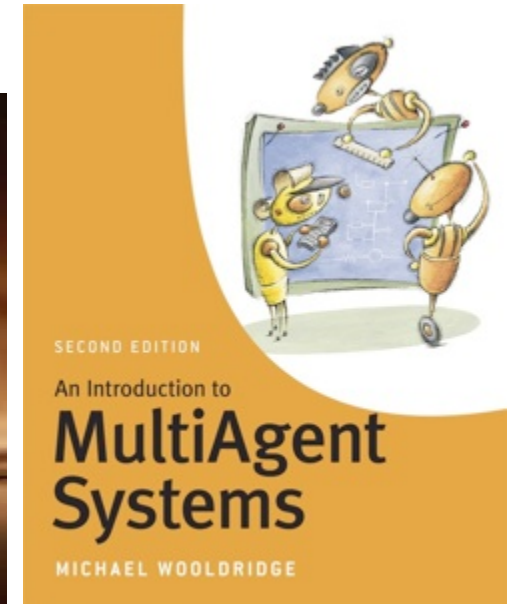
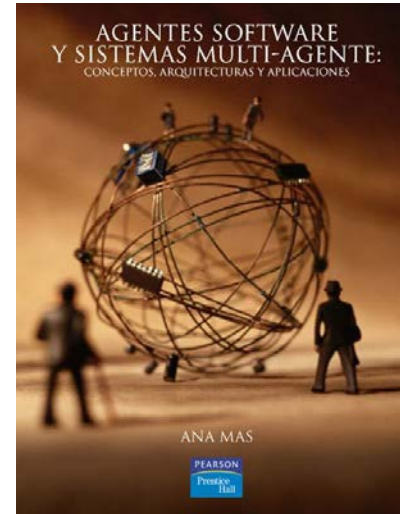
- A greater understanding of how agents, databases and information systems interact.
 - Development of benchmarks for agent system performance and efficiency.
 - New web standards that enable structural and semantic description of information.
 - Creation of common ontologies, thesauri, or knowledge bases.
-

Preconditions for wide-scale usage

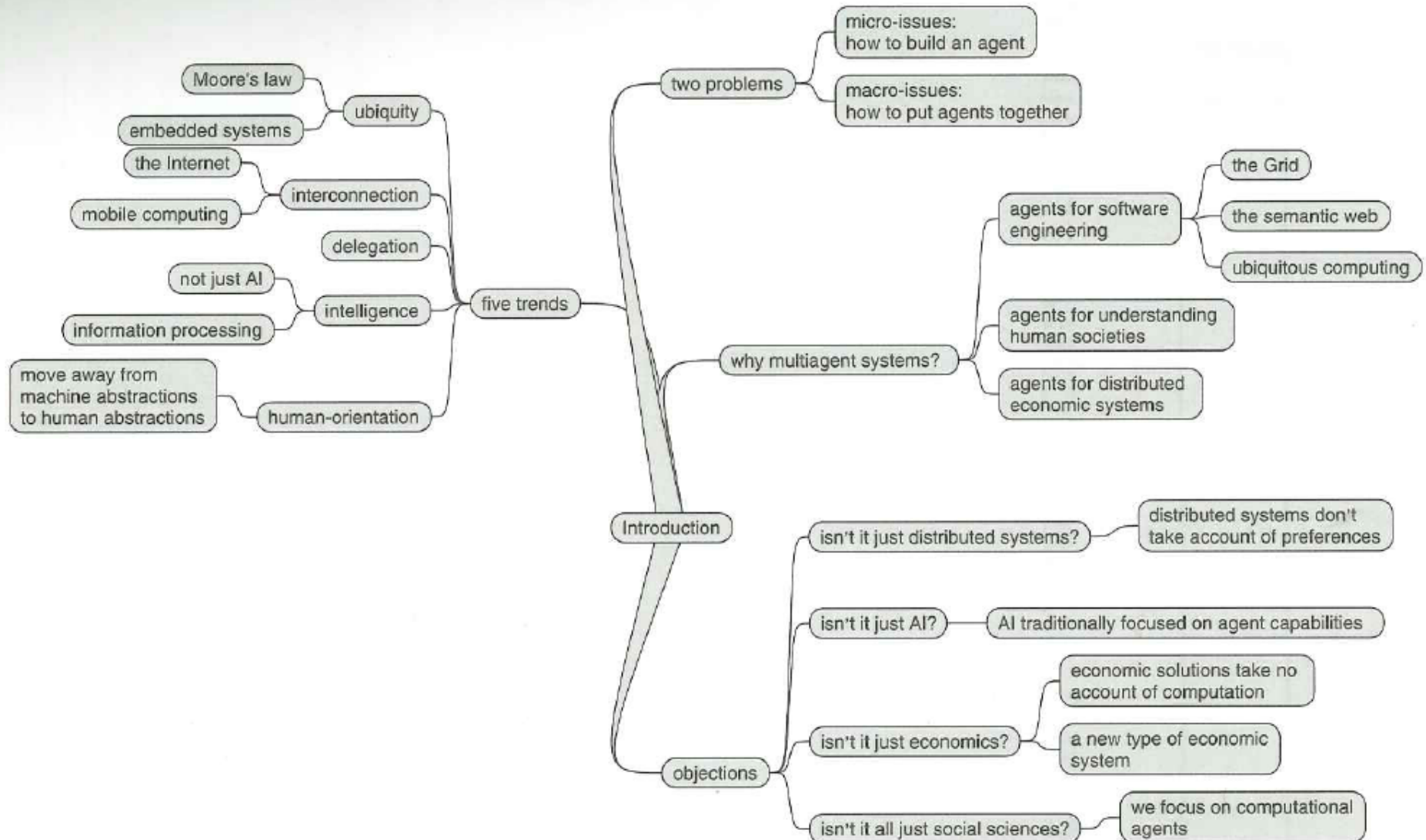
- Software agent technology is better than other ones.
 - There are enough trained, skillful persons to develop applications with software agent technology.
 - Development tools are easy to use and powerful enough to develop applications.
 - Technology is generally known.
 - Including top-level management.
-

Readings for this week

- M.Wooldridge: *An introduction to MultiAgent Systems* – *chapter 1, beginning c.2*
- A.Mas: *Agentes software y sistemas multi-agente: conceptos, arquitecturas y aplicaciones* – *initial part of chapter 1*
- *Pitfalls of Agent-Oriented Development* (M.Wooldridge, N.Jennings)
- *Agent Technology: Computing as Interaction*
 - ❑ AgentLink III roadmap



Mindmap, chapter 1 (Wooldridge)



Videos (M.Wooldridge)

- Computer Science trends leading to Agent Technology.
 - Basic definition of agent and multiagent system.
 - Common objections to Agent Technology.
 - Objects vs. Agents.
-